Chapitre 3 : Héritage

L'objectif de cette fiche est d'introduire une notion fondamentale de la Programmation Orientée Objet : l'**héritage** (*inheritance*). Ce mécanisme permet de définir une nouvelle classe à partir d'une classe existante, en *héritant* de toutes les caractéristiques (attributs et méthodes) de la classe d'origine, tout en pouvant en ajouter de nouvelles. L'héritage est une notion puissante de la POO, qui permet à la fois la réutilisation de code, une meilleure mise en facteur du code, un haut niveau d'abstraction de vos programmes et une très forte capacité d'évolution.

Problématique

Dans le cours précédent vous avez (re)découvert cet animal merveilleux qu'est le Pangolin. Pour rappel, voici une classe permettant de le représenter :

```
class Pangolin {
    private String nom;
    private int nbEcailles;

    public Pangolin(String nom, int nbEcailles) {
        this.nom = nom;
        this.setNbEcailles(nbEcailles);
    }

    public int getNbEcailles() {
        return nbEcailles;
    }

    public void setNbEcailles(int nb) {
        if (nb <= 0) {
            throw new IllegalArgumentException("Le nombre d'écailles doit
être strictement positif !");
        }
        this.nbEcailles = nb;
    }

    public void crier() {
        System.out.println("Gwwark Rhââgn Bwwikk"); // Cri du pangolin
    }
}
```

Une espèce particulière de Pangolin est le « Pangolin à longue queue », appendice qu'il utilise pour saisir des branches (véridique). Comparons ces deux animaux:

- Tous deux possèdent un grand nombre de **propriétés communes**: un nom, un nombre d'écailles et le fait de crier ;
- Comme son nom l'indique, le Pangolin a longue queue possède **en plus** une queue ;
- Même si leur cri est très proche, l'expert saura reconnaître celui **spécifique** de l'animal à queue.

Naturellement, nous brûlons d'envie d'écrire une classe `PangolinALongueQueue` ! Pour commencer, voici deux solutions naïves:

1. Une première solution serait de copier la classe Pangolin, en y ajoutant un attribut `longueurQueue` et en adaptant le constructeur et la méthode `crier()`. Par pudeur cette classe est ici masquée, mais elle ressemblerait à ça:

   Solution naïve 1

```
class PangolinALongueQueueNaif1 {
    private String nom;                         // Commun
    private int nbEcailles;                     // Commun
    private int longueurQueue;                  // Spécifique

    public PangolinALongueQueueNaif1(String nom, int nbEcailles, int
longueurQueue) {
        this.nom = nom;                         // Commun
        this.setNbEcailles(nbEcailles);         // Commun
        this.longueurQueue = longueurQueue; // Spécifique
    }

    public int getNbEcailles() {                // Commun
        return nbEcailles;
    }

    public void setNbEcailles(int nb) {         // Commun
        if (nb <= 0) {
            throw new IllegalArgumentException("Le nombre d'écailles
doit être strictement positif !");
        }
        this.nbEcailles = nb;
    }

    public int getLongueurQueue() {             // Spécifique
        return longueurQueue;
    }

    public void crier() {                       // Spécifique
        System.out.println("Qeyyyoouuuu Gwwark Rhââgn Bwwikk");
    }
}
```

   Comme vous pouvez le constater, ce code est très redondant ce qui est loin d'être satisfaisant. De plus, imaginez le travail (et la maintenance !) si vous devez un jour ajouter une nouvelle sorte de Pangolin ou préciser la couleur des écailles de ces animaux... Enfin, cette solution naïve ne rend pas compte du fait qu'un Pangolin à longue queue n'est, somme toute, qu'un cas particulier de Pangolin. Si par exemple il vous prend l'envie d'écrire un traitement qui prend un Pangolin quelconque en paramètre, eh bien il vous faudrait écrire en fait deux méthodes : une qui prend un `Pangolin` en paramètre et une autre qui prend un `PangolinALongueQueue` en paramètre !

2. Une deuxième solution serait de ne garder que la classe `Pangolin` d'origine, et de lui ajouter un attribut `longueurQueue`, qui ne serait pertinent que pour certains Pangolins. Mais alors, que signifie `getLongueurQueue()` pour les autres? De plus, il faudrait ajouter un attribut pour savoir si l'animal a une queue, et un test dans la

méthode `crier()` pour faire crier l'animal correctement selon son type.

Bien entendu, rien de tout ceci n'est acceptable. La solution va donc venir du principe d'**héritage**.

Une notion essentielle

« L'**héritage** constitue la deuxième caractéristique essentielle d'un langage orienté objet, avec l'**encapsulation** (abstraction des données) et avant le **polymorphisme** ». Bruce Eckel, *Thinking in JAVA*.

Héritage

La raison fondamentale qui permet d'avoir recours à l'héritage dans notre petit exemple est qu'un Pangolin à longue queue **- EST -** avant tout un Pangolin ! Nous allons définir une nouvelle classe `PangolinALongueQueue` qui **hérite** de la classe `Pangolin`.

Avec l'héritage :

- La nouvelle classe `PangolinALongueQueue` possédera les **mêmes propriétés**, attributs et méthodes, que la classe `Pangolin` : ces propriétés sont *héritées*.
- Il est possible de lui **ajouter** des attributs ou des méthodes supplémentaires.

En JAVA, le mot clé traduisant l'héritage entre classes est `extends`. Voici donc une première version de la classe `PangolinALongueQueue` :

```
class PangolinALongueQueue extends Pangolin {
    private int longueurQueue;

    public PangolinALongueQueue(String nom, int nbEcailles, int
longueurQueue) {
        // [À COMPLÉTER...]
        this.longueurQueue = longueurQueue;
    }

    public int getLongueurQueue() {
        return longueurQueue;
    }
}
```

Nous pouvons maintenant créer notre premier objet à longue queue :

```
class Test {
    public static void main(String [] args) {
        Pangolin gerard = new Pangolin("Gérard", 1542);
        PangolinALongueQueue rantanplan = new
PangolinALongueQueue("Rantanplan", 1966, 28);

        System.out.println(gerard.getNom() + " a " + gerard.getNbEcailles()
+ " écailles");
```

```
        System.out.println(rantanplan.getNom() + " a lui " +
rantanplan.getNbEcailles()
        + " écailles, et une queue de " + rantanplan.getLongueurQueue() + "
cm");

        // Ceci affiche:
        //    Gérard a 1542 écailles
        //    Rantanplan a lui 1966 écailles, et une queue de 28 cm

        // Crions un peu !
        gerard.crier();       // Gwwark Rhââgn Bwwikk
        rantanplan.crier();   // Gwwark Rhââgn Bwwikk
}
```

Plusieurs choses à remarquer :

- Du fait de l'héritage entre classes, l'objet `rantanplan` possède bien un nom et un nombre d'écailles, deux méthodes accesseurs sur ces attributs et une méthode `crier` ; Ces attributs et ces méthodes sont *hérités*.
- Il a en plus une queue ;

Par contre, même s'il peut bien crier, `rantanplan` crie toujours comme `gérard` ! En effet, il hérite simplement la méthode définie dans la classe `Pangolin`. Tout n'est donc pas encore parfait. Mais une nouvelle classe a été crée à moindre coût, en réutilisant au maximum du code existant.

Terminologie

Avant de continuer, introduisons un peu de terminologie :

- L'ensemble des classes liées par une relation d'héritage s'appelle une **hiérarchie de classes** ;
- Une **classe mère** ou **super-classe** est une classe dont on hérite. Réciproquement, une **classe fille** ou **sous-classe** est une classe qui hérite d'une autre classe. Ici, `Pangolin` est la mère de `PangolinALongueQueue`, qui est sa fille.
- Une classe fille **spécialise** ou **étend** sa classe mère. Un `PangolinALongueQueue` est un *cas particulier* de `Pangolin`. Mais, important, c'EST avant tout un `Pangolin` !
- Une classe mère **généralise** les notions communes à sa/ses classe(s) fille(s). Ici `Pangolin` regroupe les caractéristiques partagées par les Pangolins « communs », ceux à longue queue, ceux à écailles tricuspides, etc.

Hiérarchies de classes

Une classe peut avoir plusieurs filles, c'est-à-dire être héritée par plusieurs autres classes. De plus ces filles peuvent elles-mêmes avoir des filles. Se forment alors des *hiérarchies de classes* permettant de représenter des familles d'objets ayant des caractéristiques communes (en haut de l'arbre) et des spécificités (en bas).

La figure ci-dessous donne l'exemple d'une hiérarchie de classes pour quelques animaux. La notation utilisée est ici UML (*Unified Modeling Langage*).

Il n'y a pas de limitation du nombre de niveaux dans la hiérarchie d'héritage. Les attributs et méthodes sont hérités au travers de tous les niveaux. Ainsi, une méthode définie dans une classe est héritée par ses petites-petites-petites filles.

Et qu'y a-t-il au-dessus ?

En fait, en Java, toute classe qui n'a pas explicitement de classe mère hérite *implicitement* d'une super-classe nommée `Object`. Elle est donc la racine commune de TOUTES les (hiérarchies de) classes JAVA existantes.

Récréation

Qu'on se le dise, l'`Homme` est en fait cousin du `Pangolin` (si, si...).

La fin de cette fiche reviendra plus en profondeur sur cette `super-classe Object`.

Héritage simple (et pas multiple)

Conceptuellement, il est assez courant de pouvoir considérer un objet suivant différents point de vue. Par exemple un `Pangolin` est un `Animal`, mais pourrait aussi être vu comme un cas particulier de `Peluche` ou d'`ObjetDeDerision`. Il serait donc parfois pertinent d'hériter de *plusieurs* classes, pour hériter de leurs différentes caractéristiques. On parle alors d'**héritage multiple**. Pourtant ce n'est pas un problème simple, ni en conception ni sur le plan technique.

- En langage de modélisation Orientée Objet *UML*, l'héritage multiple est parfaitement défini ;
- Certains langages de programmation l'autorisent, comme C++ ou Python. Il faut alors mettre en place des mécanismes techniques pas toujours simples en pratique ; mais il n'y a pas de solution évidente et n'ayant que des avantages. Ceux qui veulent s'en persuader pourront se documenter sur le « problème du diamant » (*diamond problem*).
- D'autres langages ont choisi de ne faire que de l'héritage simple. C'est le cas de Java, d'ObjectiveC ou de C# par exemple. Le principe est alors de revoir la conception des diagrammes de classes pour contourner le problème.

Nous retiendrons donc qu'en JAVA, il n'y a qu'une seule Maman :

En JAVA, l'héritage multiple n'est *simplement pas possible*. **Une classe ne peut hériter que d'une seule classe** !

Redéfinition de méthodes

Une sous-classe peut ajouter de nouvelles caractéristiques (méthodes et attributs) à celles héritées de sa classe mère. Mais elle peut aussi redéfinir des méthodes héritées pour adapter – spécialiser – leur comportement.

- **Redéfinir** une méthode signifie écrire dans la sous classe une méthode dont la *signature* (nom, type de retour et paramètres) est identique à celle d'une des méthodes héritées ;
- Lorsqu'une méthode est redéfinie, elle *remplace* la méthode héritée. C'est la nouvelle définition qui est exécutée et pas celle de la classe mère.

Pour nos chers animaux, observant qu'un Pangolin à longue queue ne crie pas comme une Pangolin sans queue, nous allons redéfinir la méthode `crier()` dans la classe `PangolinALongueQueue` :

```
class PangolinALongueQueue extends Pangolin {
    [...]   // Même code que précédemment

    @Override
    public void crier() {
        System.out.println("Qeyyyoouuuu Gwwark Rhââgn Bwwikk");
    }
}
```

Observons :

```
System.out.println(gerard.getNbEcailles());        // 1542
System.out.println(rantanplan.getNbEcailles());    // 1966

gerard.crier();         // Gwwark Rhââgn Bwwikk
rantanplan.crier();     // Qeyyyoouuuu Gwwark Rhââgn Bwwikk
```

- La méthode `getNbEcailles()` invoquée sur l'objet `rantanplan` est celle héritée depuis la classe `Pangolin`; donc la même que pour `gerard`, définie dans la classe `Pangolin`.
- Par contre, la méthode `crier()` a été redéfinie dans la classe fille. C'est donc cette nouvelle définition qui est invoquée sur `rantanplan`. Le comportement est maintenant bien spécifique !

Redéfinition avec réutilisation

Lors d'une redéfinition, il est possible d'appeler la méthode héritée que l'on est en train de réécrire au moyen du mot clé `super` :

```
    @Override
    public void crier() {
        System.out.print("Qeyyyoouuuu "); // partie spécifique
        super.crier();                    // partie commune (réutilisation)
    }
```

De la même manière que `this` est la référence à l'instance sur laquelle la méthode est invoquée, `super` permet de désigner la classe mère. L'usage de `super` pour désigner un champ de la super-classe peut être fait à n'importe quel endroit dans le code de la sous-classe.

Redéfinition *vs* surcharge

Attention

Ne pas confondre **redéfinition** (*override*; réécriture d'une méthode héritée) et **surcharge** (*overload*; ajout d'une nouvelle méthode de même nom) !

Rappel :

- **Surcharger** une méthode (dans une classe, ou dans une sous classe) signifie écrire une *nouvelle* méthode qui a le même nom qu'une méthode déjà existante, mais dont le nombre et/ou le type des paramètres diffère. Surcharger *ajoute* une nouvelle méthode à la classe.
- **Redéfinir** une méthode dans une sous-classe signifie écrire une méthode qui a exactement la même signature qu'une méthode héritée. La méthode redéfinie *remplace*, dans la clase fille, la méthode héritée.

Détails sur la différence entre surcharge et redéfinition
Supposons que, **à la place de la méthode** `crier()` précédente, nous avions écrit :

```
class PangolinALongueQueue extends Pangolin {
    [...]   // Même code que précédemment, SAUF crier()!

    // Cri en conditions potentiellement dégradées
    public void crier(boolean piedSurLaQueue) {
        // Partie spécifique
        if (piedSurLaQueue)
            System.out.print("Ouap! On m'a marché sur la queue! ");
        else
            System.out.print("Qeyyyoouuuu ");
        // Cri commun de l'espèce
        super.crier();
    }
}
```

Alors :

```
gerard.crier();             // Gwwark Rhââgn Bwwikk
rantanplan.crier(true);     // Ouap! On m'a marché sur la queue! Gwwark
Rhââgn Bwwikk
rantanplan.crier(false);    // Qeyyyoouuuu Gwwark Rhââgn Bwwikk
rantanplan.crier();         // Gwwark Rhââgn Bwwikk
```

Dans cet exemple, la classe `PangolinALongueQueue` surcharge la méthode `crier` : elle ne redéfinit (remplace) PAS la méthode héritée de `Pangolin` mais en crée une nouvelle, de même nom mais ayant une autre *signature* ! La classe possède donc DEUX méthode `crier` différentes.

L'appel à `rantanplan.crier()` (sans paramètre) est correct car la méthode existe par héritage. Mais son résultat n'est pas le résultat voulu.

Une solution serait au final de rajouter la méthode spécifique au Pangolin à longue queue (surcharge), mais d'également bien redéfinir la méthode héritée avec le comportement adéquat :

```
class PangolinALongueQueue extends Pangolin {
    [...]   // Même code que précédemment

    // SURCHAGE: Cri en conditions potentiellement dégradées
    public void crier(boolean queue) {
        // Partie spécifique
        if (queue)
            System.out.print("Ouap! On m'a marché sur la queue! ");
        else
            System.out.print("Qeyyyoouuuu ");
        // Cri commun de l'espèce
        super.crier();
    }

    // REDEFINITION de la méthode commune à tous les Pangolins
    @Override
    public void crier() {
        this.crier(false);
    }
}
```

Annotations

L'*annotation* `@Override` précédant la redéfinition d'une méthode permet de notifier au compilateur qu'il s'agit bien de la redéfinition d'une méthode héritée.

Ceci évite de faire une surcharge par inadvertance (le programme ne compile plus), ce qui est une erreur assez commune mais potentiellement longue à détecter. Il est conseillé de toujours utiliser les annotations.

Remarque sur le type de retour pour la surcharge

Il n'est pas possible de surcharger une méthode en ne changeant que son type de retour. Les paramètres doivent être différents, en nombre et/ou en type. Par exemple `public void crier(boolean)` ou `public String crier(float, float)` sont valides, mais pas `public String crier()`.

Remarque sur le type de retour pour la redéfinition

Lors d'une redéfinition, la nouvelle méthode doit avoir le même nom et exactement les mêmes paramètres que la méthode redéfinie. Pour être précis, signalons que, depuis Java 1.5, le type de retour de la redéfinition peut être une *sous-classe* du type de retour de la méthode redéfinie.

Exemple

```
public class A1 { ... }

            public class A2 extends A1 { ... }

            public class B1 {
```

```
            public A1 uneMethode ( ... ) { ... }
        }

        public class B2 extends B1 {
            // redéfinition valide , car A2 est une sous - classe de A1
            public A2 uneMethode ( ... meme parametres que dans A1 ...
) { ... }
        }
```

Remarque sur la visibilité lors d'une redéfinition

La visibilité d'une méthode redéfinie doit être **égale ou moins restrictive** que celle de la méthode héritée :

- La redéfinition d'une méthode `public` ne peut être que `public` ;
- Celle d'une méthode `protected` peut être `protected` ou `public` ;
- Celle d'une méthode sans visibilité (paquetage) peut être paquetage, `protected` ou `public` ;
- Enfin une méthode `private` ne peut pas être redéfinie puisqu'elle n'est pas accessibles aux filles ! (implicitement, elle est `final`)

Par exemple, vous aurez peut-être rencontré le problème en essayant de redéfinir `String toString()` (sans visibilité). Elle doit être `public` !

Construction

Les constructeurs d'une classe servent à initialiser ses attributs lors de la création d'un objet. Avec l'héritage, il faut aussi initialiser les attributs hérités. Ceci se fait en invoquant les constructeurs de la classe mère, avec les paramètres adéquats, au moyen du mot-clé super() :

```
class Pangolin {
    private String nom;
    private int nbEcailles;

    public Pangolin(String nom, int nbEcailles) {
        this.nom = nom;
        this.setNbEcailles(nbEcailles);
    }

    [...] // reste de la classe
}

class PangolinALongueQueue extends Pangolin {
    private int longueurQueue;

    public PangolinALongueQueue(String nom, int nbEcailles, int
longueurQueue) {
        super(nom, nbEcailles);                // appel au constructeur de
la mère
        this.longueurQueue = longueurQueue;    // initialisation(s)
spécifique(s)
    }

    [...] // reste de la classe
}
```

Il faut donc initialiser les attributs hérités de la mère, puis ceux spécifiques à la classe.

- L'appel au constructeur de la mère se fait avec `super(...)`. Si la super classe possède plusieurs constructeurs, le nombre et type des arguments déterminent lequel est invoqué.
- L'appel de `super(...)` doit toujours être **la première instruction** dans le corps du constructeur.
- Si ce n'est pas le cas, alors le compilateur insère implicitement en 1ère ligne l'appel `super()` au constructeur par défaut (sans paramètres) de la mère. Si celui-ci n'existe pas, il y a une erreur.

Il est toujours recommandé d'initialiser les attributs hérités via `super(...)` et jamais directement. Outre la réutilisation du code existant, ceci permet d'assurer l'intégrité des paramètres qui est garantie par la mère.

En fait, le principe d'**encapsulation** doit être respecté, même entre classes d'une même hiérarchie !

Constructeur par défaut

Jusque-là nous avions vu que si une classe ne définit par explicitement de constructeur, un *constructeur par défaut* est implicitement ajouté, qui ne fait rien. En réalité ce n'est pas tout à fait vrai: il appelle le constructeur par défaut de sa classe mère !

```
class Pangolin {
    // Si AUCUN constructeur n'est défini, le constructeur par défaut est implicitement:
    public Pangolin() {
        super();
    }

    [...]   // reste de la classe
}
```

Si la classe mère n'a elle-même pas de constructeur par défaut, il y aura une erreur de compilation.

Comme toujours, le constructeur par défaut n'est généré que si aucun autre constructeur n'est défini dans la classe. Dès qu'au moins un constructeur explicite existe, c'est à vous d'écrire le constructeur sans paramètre si vous en avez besoin.

Chaînage ascendant des constructeurs

Lors de la création d'un objet, les constructeurs d'une hiérarchie de classes sont *appelés* du bas vers le haut et *exécutés* du haut vers le bas. Avec la hiérarchie ci-dessous :

- l'ordre des appels est : `PangolinALongueQueue`, `Pangolin` puis `Animal`.
- l'ordre d'exécution (des initialisations spécifiques) est donc : `Animal`, `Pangolin` puis `PangolinALongueQueue`.

```
class Animal {
```

```
    private String nom;

    // 3ème APPEL
    public Animal(String nom) {
        // super();    --> IMPLICITE!        // invoque le constructeur de
la mère
        this.nom = nom;                      // 1ère EXECUTION
    }
    [...]
}

class Pangolin extends Animal {
    private int nbEcailles;

    // 2ème APPEL
    public Pangolin(String nom, int nbEcailles) {
        super(nom);                          // invoque le constructeur de
la mère
        this.nbEcailles = nbEcailles;        // 2ème EXECUTION
    }
    [...]
}

class PangolinALongueQueue extends Pangolin {
    private int longueurQueue;

    // 1er APPEL
    public PangolinALongueQueue(String nom, int nbEcailles, int
longueurQueue) {
        super(nom, nbEcailles);               // invoque le constructeur de
la mère
        this.longueurQueue = longueurQueue;   // 3ème EXECUTION
    }
    [...]
}

[...]

PangolinALongueQueue rantanplan = new PangolinALongueQueue("Rantanplan",
1966, 28);
```

Visibilités

Il est possible de contrôler la *visibilité* des membres d'une classe (attributs et méthodes). Avec l'héritage, un nouveau qualificateur de visibilité apparaît pour contrôler ces droits au sein d'une hiérarchie de classe (à destinations des filles) : le mot clé `protected`.

Ce qui suit fait un point définitif sur les qualificateurs de visibilité en Java :

- `public` : accessible depuis le code de n'importe quel classe ;
- `private` : accessible uniquement à l'intérieur de la classe (mais pas par les filles);
- `protected` : accessible dans la classe et **dans toutes ses sous-classes**. (En JAVA, l'accès est en fait autorisé à toutes les classes du même paquetage. Ce n'est pas le cas général en POO) ;
- Si aucune visibilité n'est spécifiée, l'accessibilité par défaut est : accessible uniquement aux classes du même paquetage. En particulier, la visibilité par défaut interdit l'accès depuis le code d'une classe fille si elle n'est pas définie dans le même paquetage que sa mère !

Dans tous les exemples ci-dessus, vous aurez remarqué que les attributs étaient toujours de visibilité `private`. Ceci veut dire que, si un objet `PangolinALongueQueue` **possède bien** un attribut `nbEcailles`, le code de la classe fille n'y a par contre **pas accès directement** !

```
[...]

rantanplan.nbEcailles;       // INTERDIT! (private dans la mère)
rantanplan.getNbEcailles()   // Autorisé, la méthode est publique
```

Cela peut paraître peu pratique. Mais en fait déclarer `private` les attributs, même dans une classe mère, est en général une bonne pratique.

Il s'agit encore et toujours du principe d'**encapsulation** des données, qu'il convient d'appliquer aux sous-classes comme aux classes extérieures à une hiérarchie. Selon ce principe, les données propres à un objet ne sont accessibles qu'au travers de méthodes. Rappelons deux avantages :

- Sécurité des données : passer par des méthodes pour modifier l'état d'un objet permet de garantir l'intégrité de ce nouvel état. En cas de problème, la méthode peut générer une erreur (en Java, typiquement, en levant une exception).
- Masquer l'implémentation : la structure de donnée de la classe peut être modifiée sans remettre en cause le code qui utilise cette classe.

La super-classe Object

Vous l'avez compris, une hiérarchie de classes est en fait un arbre dont la racine est une première classe « qui n'a pas de mère ». Dans les exemples précédents, c'était `Pangolin` (ou `Animal`).

En réalité, une classe qui ne définit pas de clause `extends` hérite *implicitement* d'une classe nommée `Object` :

```
class Tatou extends Object {  // IMPLICITE quand on écrit juste : class Tatou
    // ça change des Pangolins non?
    [...]
}
```

En conséquence TOUTE classe hérite, directement ou indirectement, de `Object`. De ce fait, les méthodes définies dans la classe `Object` peuvent être invoquées sur tous les objets d'un programme.

Quelques-unes des principales méthodes de la classe `Object` sont :

- `public String toString()` : retourne la représentation d'un objet sous forme de chaîne de caractères ;
- `public boolean equals(Object obj)` : permet de comparer deux objets *sémantiquement* (selon leur état, leurs attributs, et pas uniquement égalité des références) ;

- `public int hashCode()` : retourne un identifiant entier unique associé à un objet. Ceci est surtout utilisé pour l'insertion dans des structures de données de type table de hachage.
- `protected void finalize()` : méthode exécutée par le ramasse-miettes au moment de la destruction d'un objet qui n'est plus référencé. Peut être redéfinie afin de bien libérer certaines ressources externes (comme un fichier) même si le programmeur a oublié de le faire. Attention, `finalize` peut poser des problèmes de performances et causer des bugs, c'est pourquoi elle est dépréciée depuis Java 9. D'autres mécanismes comme les références fantômes permettent de réagir à la destruction d'un objet.

Reprenons l'exemple de `public String toString()`. Vous l'avez vu, cette méthode peut être invoquée implicitement dans différents contextes. Par exemple `System.out.println(unObjet)` est en fait `System.out.println(unObjet.toString())`.

Ceci n'est possible que du fait que tout objet possède bien cette méthode, puisque qu'elle est définie tout en haut de la hiérarchie ! Par défaut, elle retourne une chaîne composée du nom de la classe (type) de l'objet et son adresse en mémoire, par exemple : `Pangolin@13f0c45f`. Bien entendu, il est possible, et même recommandé, de **redéfinir** la méthode `toString()` dans vos classes, pour avoir un comportement plus pertinent.

Pour finir: final

Le mot clé `final` sur une méthode interdit toute redéfinition dans une classe fille :

```
class Pangolin {
    [...]

    // Si nous avions écrit ceci, le Pangolin à longue queue ne pourrait
    // pas être connu pour son cri si caractéristique. Quelle tristesse...
    public final void crier() {
        System.out.println("Gwwark Rhââgn Bwwikk"); // Cri du pangolin
    }
}
```

Enfin, le même mot-clé `final` sur une *classe* interdit cette fois l'écriture de toute sous-classe ! (Par exemple `String` est finale.)

```
 // Ici il aurait été simplement impossible de définir la classe
PangolinALongueQueue !
// (ce qui aurait été une catastrophe pour ce cours et pour la biodiversité
en général)
final class Pangolin {
        [...]
}
```

Attention à l'utilisation de cette notion. Il n'est pas toujours évident d'imaginer les évolutions possibles d'un code, en particulier les sous-classes qui pourraient être nécessaires pour un utilisateur futur.